Experiment-1

Aim: - To find the statistical functions in python (**Mean, Median, Mode, Range, Standard Deviation, Variance, Correlation**)

Introduction: -

Statistics in general, is the method of collection of data, tabulation, and interpretation of numerical data. It is an area of applied mathematics concerned with data collection, analysis, interpretation, and presentation.

1. **Mean:**
   The mean() function from Python's statistics module is used to calculate the average of a set of numeric values. It adds up all the values in a list and divides the total by the number of elements. Formula:

$$\text{Mean}(x) = \frac{\sum x}{n}$$

2. **Median:**
   The median() method calculates the median (middle value) of the given data set. This method sorts the data in ascending order before calculating the median.
   If n is even:

$$\text{Median} = \frac{(n/2)^{th} + (n/2 + 1)^{th}}{2}$$

3. **Mode:**

   The mode() function returns the value that appears most frequently in the dataset.

4. **Range:**
   Range is the difference between maximum and minimum value.
   Formula:

$$\text{Range} = \text{Maximum} - \text{Minimum}$$

5. **Standard Deviation:**
   Standard deviation is the square root of variance.
   Formula:

$$Standard\ Deviation = \sqrt{Variance}$$

6. **Variance:**
   Variance measures the spread of the data points from the mean. It is the square of standard deviation.

Formula:

$$\square\square\square\square\square\square\square\square\square = \frac{\sum(x - \bar{x})^2}{\square}$$

7. **Correlation:**

Correlation measures the strength and direction of relationship between two variables.

Krish Sendhav Untitled-1

```python
# Krish Sendhav
# 2310DMTCSE15280
import statistics
import numpy as np

data1 = [10, 20, 30, 40, 50, 20]
data2 = [15, 25, 35, 45, 55, 25]

mean = statistics.mean(data1)
median = statistics.median(data1)
mode = statistics.mode(data1)
data_range = max(data1) - min(data1)
std_dev = statistics.stdev(data1)
variance = statistics.variance(data1)
correlation = np.corrcoef(data1, data2)[0][1]

print("Mean:", mean)
print("Median:", median)
print("Mode:", mode)
print("Range:", data_range)
print("Standard Deviation:", std_dev)
print("Variance:", variance)
print("Correlation:", correlation)

```

```
Mean: 28.333333333333332
Median: 25.0
Mode: 20
Range: 40
Standard Deviation: 14.719601443879744
Variance: 216.66666666666666
Correlation: 1.0
```

Experiment-2

Aim: - To study the data visualization tools (**Pie Chart, Bar Graph, Line Graph**) using Python library **Matplotlib**.

Introduction:

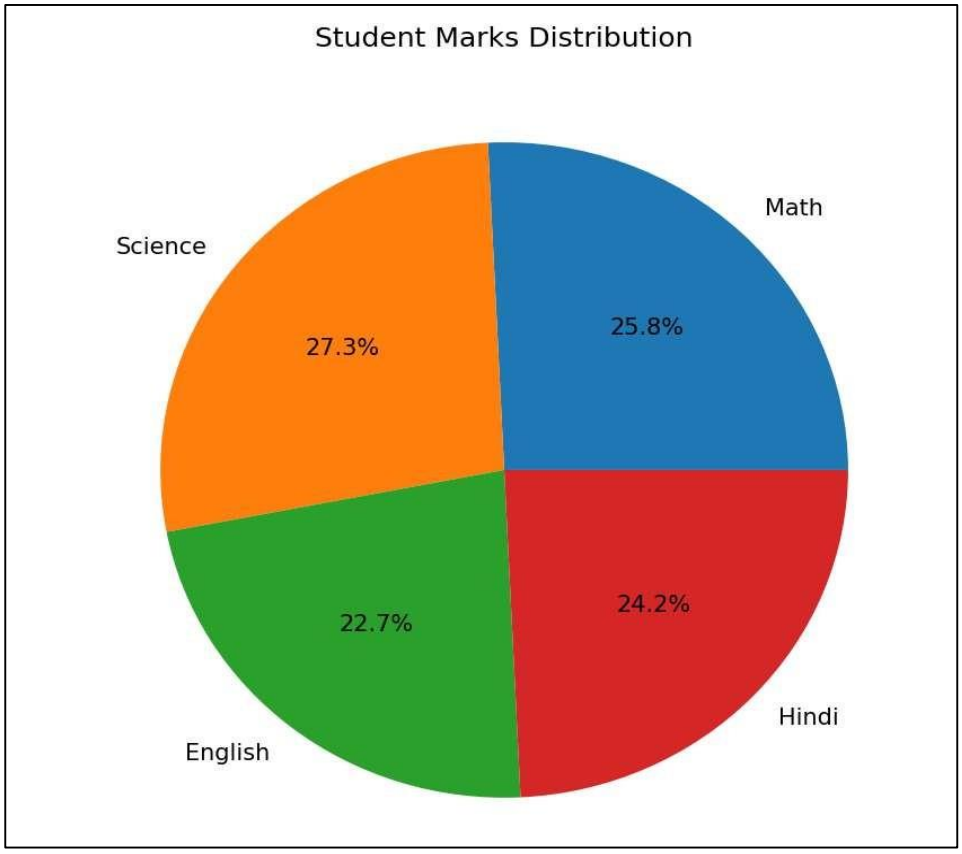
Data visualization is the graphical representation of data to understand patterns, trends, and comparisons easily.

In Python, the Matplotlib library is commonly used to create different types of graphs like pie charts, bar graphs, and line graphs.

- Pie Chart: A pie chart is used to represent proportional data as slices of a circle.

pieChart.py

```python
# Krish Sendhav
# 2310DMTCSE15280

import matplotlib.pyplot as plt

subjects = ['Math', 'Science', 'English', 'Hindi']
marks = [85, 90, 75, 80]

plt.figure(figsize=(5,4))
plt.pie(marks, labels=subjects, autopct='%1.1f%%')

plt.title("Student Marks Distribution")

plt.show()

```

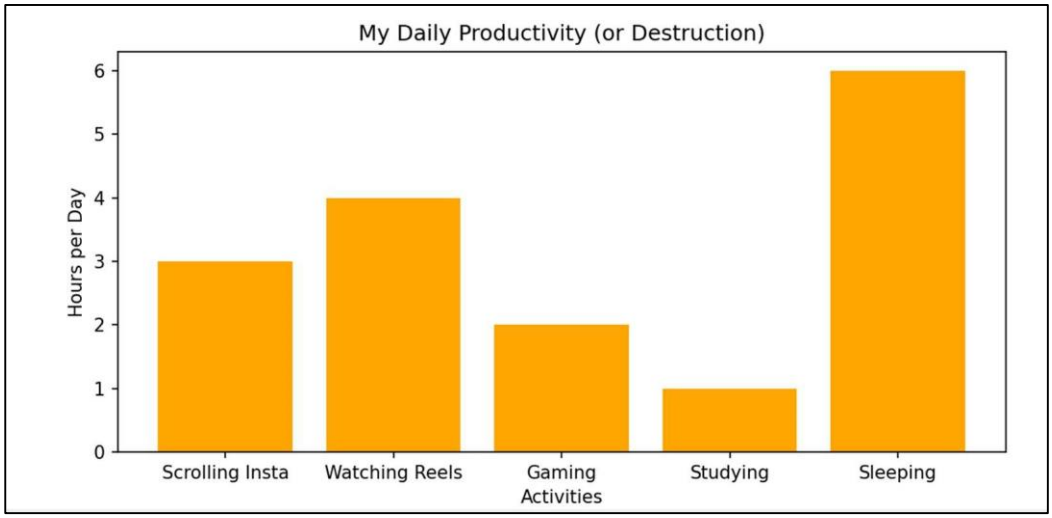


- Bar Graph: A bar graph is used to compare quantities across different categories.

pieChart.py BarGraph.py

BarGraph.py > ...

```python
# Krish Sendhav
# 2310DMTCSE15280
import matplotlib.pyplot as plt

activities = ['Scrolling Insta', 'Watching Reels', 'Gaming', 'Studying', 'Sleeping']
hours = [3, 4, 2, 1, 6]

plt.figure(figsize=(6,4))
plt.bar(activities, hours, color='orange')

plt.xlabel("Activities")
plt.ylabel("Hours per Day")
plt.title("My Daily Productivity (or Destruction)")

plt.show()
```

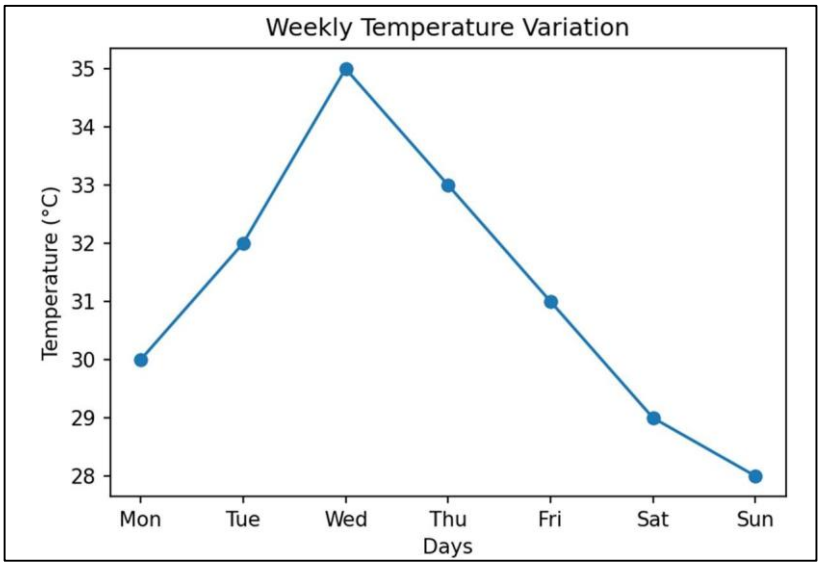


- Line Graph: A line graph is used to show changes or trends over a continuous interval

pieChart.py | BarGraph.py | lineGraph.py

lineGraph.py > ...

```python
# Krish Sendhav
# 2310DMTCSE15280
import matplotlib.pyplot as plt

days = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun']
temperature = [30, 32, 35, 33, 31, 29, 28]

plt.figure(figsize=(6,4))
plt.plot(days, temperature, marker='o')

plt.xlabel("Days")
plt.ylabel("Temperature (°C)")
plt.title("Weekly Temperature Variation")

plt.show()
```



Experiment-3

Aim: - To study **for loop**, **while loop** and understand the use of **break, continue, and pass statements** in Python.

1. **For Loop:**
   A for loop is used when the number of iterations is known in advance. It is mainly used to iterate over sequences such as lists, strings, or ranges. The loop runs a fixed number of times and executes the given block of code for each element in the sequence.

2. **While Loop:**
   A while loop is used when the number of iterations is not known beforehand. It executes a block of code as long as a specified condition remains true. The loop stops automatically when the condition becomes false. It is commonly used for condition-based execution.

3. **Break Statement:**
   The break statement is used to immediately terminate the loop, even if the loop condition is still true. It is useful when a certain condition is met and there is no need to continue further iterations.

4. **Continue Statement:**
   The continue statement is used to skip the current iteration of the loop and move to the next iteration. It does not stop the loop completely but simply ignores the remaining part of the current iteration.

5. **Pass Statement:**
   The pass statement is a null statement that does nothing when executed. It is used as a placeholder where a statement is syntactically required but no action needs to be performed. It helps in avoiding errors in empty code blocks.

(In Short)

- **For Loop** → Executes a fixed number of times using range()
- **While Loop** → Runs based on condition (i <= 10)
- **Break** → Stops loop completely at a condition
- **Continue** → Skips only that iteration (5 is skipped)
- **Pass** → Does nothing, used as placeholder

Program: -

first.py > ...

```python
# Krish Sendhav
# 2310DMTCSE15280

print("FOR LOOP:") # For Loop
for i in range(1, 11):
    print(i, end=" ")   # prints numbers from 1 to 10

print("\n\nWHILE LOOP:") # While Loop
i = 1
while i <= 10:
    print(i, end=" ")
    i += 1   # increment is necessary to avoid infinite loop

print("\n\nBREAK STATEMENT:") # Break Statement
for i in range(1, 11):
    if i == 5:
        break   # exit the loop when i = 5
    print(i, end=" ")

print("\n\nCONTINUE STATEMENT:") # Continue Statement
for i in range(1, 11):
    if i == 5:
        continue   # skip printing 5
    print(i, end=" ")

print("\n\nPASS STATEMENT:")  # Pass Statement
for i in range(1, 11):
    if i == 5:
        pass   # no action is performed
    print(i, end=" ")
```

Output: -

```
FOR LOOP:
1 2 3 4 5 6 7 8 9 10

WHILE LOOP:
1 2 3 4 5 6 7 8 9 10

BREAK STATEMENT:
1 2 3 4

CONTINUE STATEMENT:
1 2 3 4 6 7 8 9 10

PASS STATEMENT:
1 2 3 4 5 6 7 8 9 10
```

Experiment-4

Aim: To write a program in Python to predict the class of the flower based on available attributes.

Detail:

- This program uses the Iris dataset to classify flowers into different species.
- A Decision Tree model is trained using features like petal and sepal dimensions.

```
# Krish Sendhav
# 2310DMTCSE15280

# Import required libraries
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score


iris = load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = DecisionTreeClassifier()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)

sample = [[5.1, 3.5, 1.4, 0.2]]

prediction = model.predict(sample)

print("Predicted Class:", iris.target_names[prediction][0])
```

```
Accuracy: 1.0
Predicted Class: setosa
```

Experiment-5

Aim: To write a program in Python to predict if a loan will get approved or not.

Detail:

- This program predicts loan approval based on factors like income and loan amount.
- Logistic Regression is used for binary classification (approved/rejected).

```
# Krish Sendhav
# 2310DMTCSE15280

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

data = {
    "income": [25000, 40000, 50000, 60000, 20000, 80000, 120000, 70000],
    "loan_amount": [200000, 100000, 150000, 250000, 300000, 120000, 200000, 180000],
    "approved": [0, 1, 1, 1, 0, 1, 1, 1]
}

df = pd.DataFrame(data)

X = df[["income", "loan_amount"]]
y = df["approved"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = LogisticRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print(accuracy_score(y_test, y_pred))

sample = [[45000, 150000]]
result = model.predict(sample)

print("Approved" if result[0] == 1 else "Rejected")
```

```
1.0
Approved
```

Experiment-6

Aim: To write a program in Python to predict the traffic on a new mode of transport.

Detail:

- This program estimates traffic levels based on input variables like time.
- Linear Regression is used to predict continuous traffic values.

```
# Krish Sendhav
# 2310DMTCSE15280

import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

time = np.array([6, 7, 8, 9, 10, 11, 12, 13, 14, 15]).reshape(-1, 1)
traffic = np.array([20, 30, 50, 80, 60, 40, 30, 35, 45, 55])

X_train, X_test, y_train, y_test = train_test_split(time, traffic,
                                    test_size=0.2, random_state=42)
model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print(mean_squared_error(y_test, y_pred))

new_time = np.array([[16]])
prediction = model.predict(new_time)

print("Predicted Traffic:", prediction[0])
```

```
105.59230083234247
Predicted Traffic: 49.56896551724138
```

Experiment-7

Aim: To write a program in Python to predict the class of user.

Detail:

- This program classifies users into categories such as premium or normal users.
- Decision Tree algorithm is used for classification based on user data.

```
# Krish Sendhav
# 2310DMTCSE15280

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

data = {
    "age": [18, 25, 30, 22, 35, 40, 28, 50],
    "income": [10000, 25000, 40000, 15000, 50000, 60000, 30000, 70000],
    "usage_time": [1, 3, 5, 2, 6, 7, 4, 8],
    "user_class": [0, 0, 1, 0, 1, 1, 1, 1]
}

df = pd.DataFrame(data)

X = df[["age", "income", "usage_time"]]
y = df["user_class"]

X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=0.2, random_state=42)
model = DecisionTreeClassifier()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print(accuracy_score(y_test, y_pred))

sample = [[27, 35000, 4]]
prediction = model.predict(sample)

print("Premium" if prediction[0] == 1 else "Normal")
```

```
0.5
Premium
```

Experiment-8

Aim: To write a program in Python to identify the tweets which are hate tweets and which are not.

Detail:

- This program detects hate speech in tweets using text processing techniques.
- Naive Bayes classifier is used for text classification.

```
# Krish Sendhav
# 2310DMTCSE15280

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score

data = {
    "tweet": [
        "I love this product",
        "This is amazing",
        "I hate you",
        "You are so stupid",
        "What a great day",
        "Terrible experience",
        "I enjoy using this",
        "Worst thing ever"
    ],
    "label": [0, 0, 1, 1, 0, 1, 0, 1]
}

df = pd.DataFrame(data)

X = df["tweet"]
y = df["label"]

vectorizer = CountVectorizer()
X_vectorized = vectorizer.fit_transform(X)
```

```
X_train, X_test, y_train, y_test = train_test_split
(X_vectorized, y, test_size=0.2, random_state=42)

model = MultinomialNB()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print(accuracy_score(y_test, y_pred))

sample = ["I hate this so much"]
sample_vec = vectorizer.transform(sample)

prediction = model.predict(sample_vec)

print("Hate Tweet" if prediction[0] == 1 else "Not Hate Tweet")
```

```
0.5
Hate Tweet
```

Experiment-9

Aim: To write a program in Python to predict the age of the actors.

Detail:

- This program predicts age based on features like experience or other inputs.
- Linear Regression is used to estimate numerical values.

```
# Krish Sendhav
# 2310DMTCSE15280

import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error

experience = np.array([1, 2, 3, 5, 7, 10, 12, 15]).reshape(-1, 1)
age = np.array([20, 22, 25, 30, 35, 40, 45, 50])

X_train, X_test, y_train, y_test = train_test_split
 (experience, age, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print(mean_absolute_error(y_test, y_pred))

new_data = np.array([[8]])
prediction = model.predict(new_data)

print("Predicted Age:", prediction[0])
```

```
0.6628308400460252
Predicted Age: 35.95512082853855
```

Experiment-10

Aim: Mini project to predict the time taken to solve a problem given the current status of the user.

Detail:

- This project predicts the time required based on user experience and difficulty level.
- Regression techniques are used to estimate the solving time.

```
# Krish Sendhav
# 2310DMTCSE15280

import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

experience = np.array([1, 2, 3, 4, 5, 6, 7, 8]).reshape(-1, 1)
difficulty = np.array([1, 2, 3, 2, 3, 4, 5, 4]).reshape(-1, 1)
time_taken = np.array([30, 45, 60, 50, 70, 90, 120, 110])

X = np.hstack((experience, difficulty))
y = time_taken

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print(mean_squared_error(y_test, y_pred))

new_data = np.array([[5, 3]])
prediction = model.predict(new_data)

print("Predicted Time:", prediction[0])
```

```
18.385850178359075
Predicted Time: 75.17241379310344
```